

CS571 Programming Languages

Spring 2026

11. Object-Oriented Programming



Agenda

- Key Elements of OOP:
 - Encapsulation
 - Inheritance
 - Subtyping
- Implementation of OOP:
 - Subtype Polymorphism

Encapsulation

- The **data** and **computation** are combined into object
(object = data + computation)
 - **More convenient**: collect related information together, rather than distributing it
 - **Centralizes and regulates access to data**: if there is an error in the program that corrupts object data, we need to look for the error only within the computations associated with the objects
 - **Permitting extension of new objects via inheritance**: more later
 - **Hide irrelevant details**: using visibility modifiers, such as public, private, protected

Class and Objects

- **Class** is a type and includes data and operations together
- An **object** is an **instance** of class
 - Objects are declared to be of a particular class exactly as variables are declared to be of a particular type in C

Fields and Methods

- **Member variables / fields**
 - **Instance variables**: each object has its own copy of instance variables
 - **Class (static) variables**: across **all objects** of a class – similar to global variables
- Each object includes a set of functions called **methods**.
 - Similar to ordinary procedures and functions
 - Can automatically access the object's data.



Example: Complex Numbers (Java)

```
public class Complex{
    private double re, im;
    public Complex() { re = 0; im = 0;}
    public Complex (double realpart, double imagpart) { re = realpart; im = imagpart; }
    public double realpart() { return re;}
    public double imaginarypart() { return im;}
    public Complex add (Complex c)
    {   return new Complex(re + c.realpart(), im + c.imaginarypart());}
    public Complex multiply (Complex c)
    {   return new Complex(re * c.realpart() - im * c.imaginarypart(),
                           re * c.imaginarypart() + im * c.realpart()); }
}
```



Example: Complex Numbers (Java)

```
import java.io.*;
public class Main{
    public static void main(String[] args){
        Complex z, w;
        z = new Complex (1,2);
        w = new Complex (-1,1);
        z = z.add(w);
        System.out.println(z.realpart());
        System.out.println(z.imaginarypart());
        z = z.add(w).multiply(z);
        System.out.println(z.realpart());
        System.out.println(z.imaginarypart());}
}
```



What to Remember

- An **object** is a run-time instance of a **class**. Each class can have many instances. All objects from the same class share the **methods**, but each of them have different “internal” states, stored in the **fields**
- Instantiation is achieved by the **new** expression. **Constructors** are used here, so that an object can be created with initial values. Constructors **do not** have return values. Each class may have **multiple** constructors, a case of overloading
- Different objects are “hooked up” together via **method invocation**
- The entire program starts from the **main** method, and only one class can have it
- **Object types** are class names



Agenda

- Key Elements of OOP:
 - Encapsulation
 - Inheritance
 - Subtyping
- Implementation of OOP:
 - Subtype Polymorphism

Inheritance

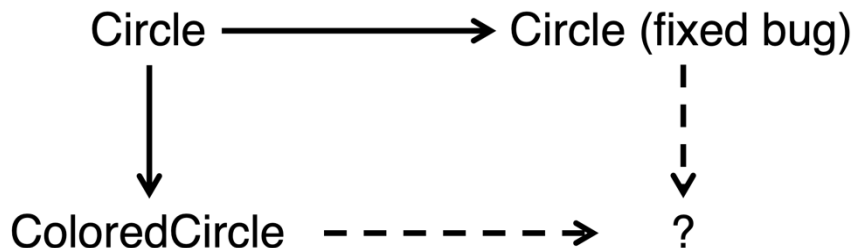
- Allows **code reuse**
 - If you plan to write some code for “GradStudent” and know your friend has written the code for “Student”, why not just use your friend’s code as a base and modify based on it, rather than rewrite from scratch?
- Inheritance in essence is a “**code + code = a bigger piece of code**”-kind of operation
 - It is related to **subtyping**, but these two concepts are distinct, and some OO languages actually treat them orthogonally.

Motivation of Inheritance

```
class Circle implements Shape {  
    double radius;  
    public double area() {return 3.14*radius*radius};  
    public int edges() {return 1};  
}
```

How to implement ColoredCircle?

Option 1: copy-and-paste

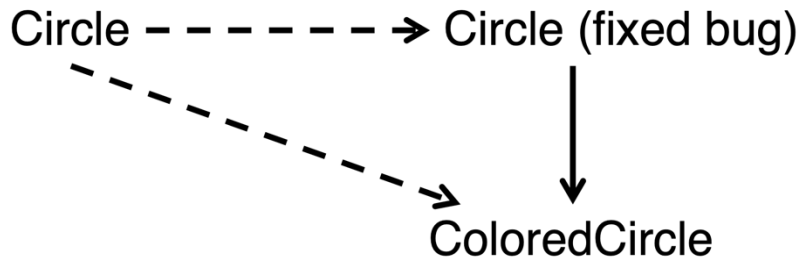


Inheritance

```
class ColoredCircle extends Circle {  
    private Color color;  
    Color getColor {return color};  
    // methods area, edges are inherited from Circle  
}
```

How to implement ColoredCircle?

Option 2: inheritance



Two Uses of Inheritance

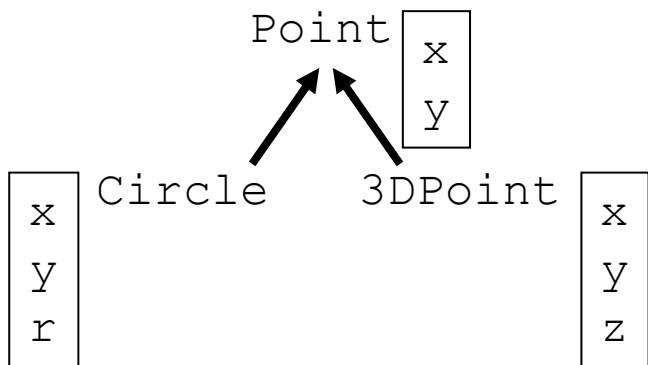
- **Extension**

- A subclass can **add** data members and member functions of its own

- **Redefinition (method overriding)**

- An implementation of a method in a subclass supersedes the implementation of the same method in the base class
- A derived class is **more specific** than a base class

Inheritance Example



```
public class Point{  
    private int x, y;  
    . . .  
};
```

```
public class Circle extends Point{  
    private double r;  
    . . .  
};
```

```
public class 3DPoint extends Point{  
    private int z;  
    . . .  
};
```



Inheritance in C++

- `class B: public A`
 - All `public/protected` members of A remain `public/protected` in B. Private members of A are unavailable in B
- `class B: private A`
 - B has `private` access to the `public` and `protected` members of A
- `class B: protected A`
 - The `public` and `protected` parts of the base class become protected in the derived class



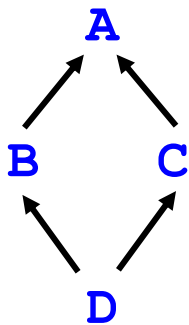
Single Inheritance

- **Single inheritance**: each subclass inherits from only one superclass
 - All the examples you have seen so far are single inheritance
 - The only mode for Java

Multiple Inheritance

- **Multiple inheritance:** a class may inherit from two or more superclasses

- C++ provides



- Difficult to resolve references to methods.
- What happens if B, but not C, redefines a method of A?

- Java has **multiple interface inheritance** which is almost as powerful, and much easier to implement.
- Multiple inheritance is out of scope of this class



Agenda

- Key Elements of OOP:
 - Encapsulation
 - Inheritance
 - Subtyping
- Implementation of OOP:
 - Subtype Polymorphism

Subtyping

- "Is-a" Property: sometimes, every value of type B is of type A
 - e.g., a `Circle` is always a `Shape`
- We say B is a **subtype** of A, meaning "every object that satisfies interface of B also satisfies interface of A"
- Liskov Substitution Principle: If B is a subtype of A, then objects of type A in a program may be replaced with objects of type B without altering any of the desirable properties (like correctness) of that program



More on Liskov Substitution Principle

- LSP means is that it is always OK to assign $x = y$ if y is a subtype of x , without making your program crash at run time
- It just so happens that Java inheritance hierarchy satisfies the same relation, but this is more of a “coincidence” than a necessity
 - Many OO programming languages do not fit into this “coincidence”, including C++.
 - Interested students should read “Inheritance is Not Subtyping” (POPL 1989)
 - Another good reference is “Data Abstraction and Hierarchy” (OOPSLA 1987)

C++: Inheritance != Subtyping

```
class A {  
    public: void f();  
    ...};  
class B: private A {  
    ...}; // no public redefinition of f()  
  
A* x = new A();  
B* y = new B();  
x->f(); // legal  
y->f(); // illegal – inheritance != subtyping
```

Java Interface

- An interface may only contain **methods with empty bodies** and **constant declarations** (variables declarations which are declared to be both **static** and **final**).
- e.g. A bicycle's behavior:

```
interface Bicycle {  
    void changeCadence(int newValue);  
    void changeGear(int newValue);  
    void speedUp(int increment);  
    void applyBrakes(int decrement);  
}
```

Java Interface

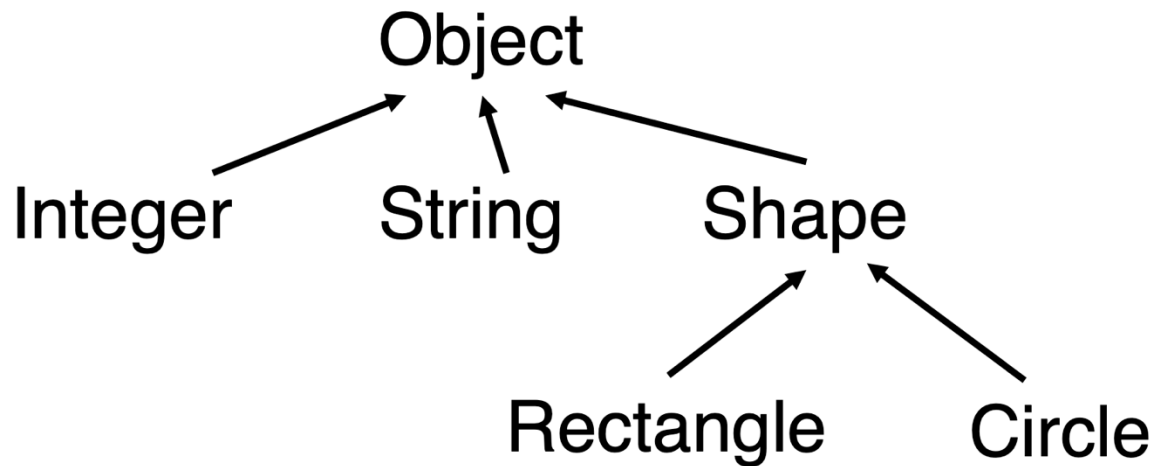
- Interfaces cannot be instantiated – they can only be implemented by classes or extended by other interfaces
- A (non-abstract) class that **implements an interface** must implement **all** of the interface's methods
 - It is possible, however, to define a class that does not implement all of the interface methods, provided that the class is declared to be abstract
- The class implementing the interface is a subtype of the interface type

```
abstract class X implements Y
{ // implements all but one method of Y }
class XX extends X
{ // implements the remaining method in Y }
```

Subtyping Relation

- We write $B \leq A$ to mean B is a subtype of A
 - A is called a supertype or base type
- Relation $\leq$ defines a **partial order** on types
 - Reflexibility: $\forall T: T \leq T$
 - Antisymmetry: $\forall T_1, T_2: T_1 \leq T_2 \wedge T_2 \leq T_1 \Rightarrow T_1 = T_2$
 - Transitivity: $\forall T_1, T_2, T_3: T_1 \leq T_2 \wedge T_2 \leq T_3 \Rightarrow T_1 \leq T_3$

Subtype Lattice Example



Subtyping: Summary

- Who defines the subtype relationship? The programmers!
 - Java: B is a subtype of A iff
 - (1) B is a subclass of A; OR
 - (2) B implements A as an interface.
 - C++: B is a subtype of A iff B is a subclass of A **AND the inheritance is public**
 - Subtyping is reflexive
 - Subtyping is transitive
- Who uses the subtyping relationship? The Compiler in its type checking process
 - Subtype principle: if I expect a value of a type, it is ok the value at runtime is a subtype of that type

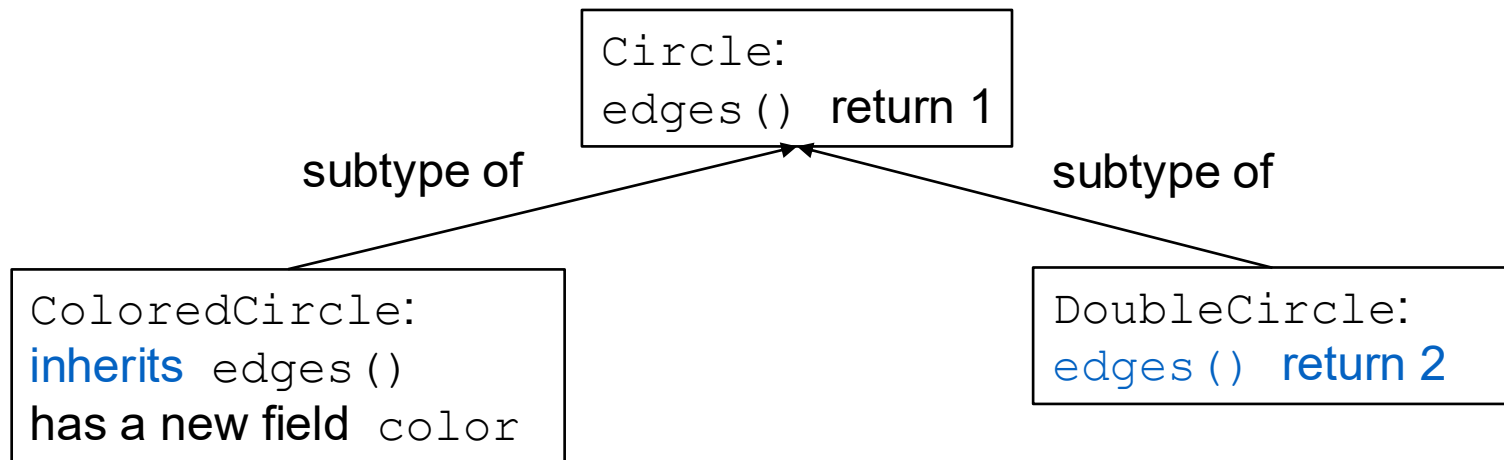
Agenda

- Key Elements of OOP:
 - Encapsulation
 - Inheritance
 - Subtyping
- Implementation of OOP:
 - Subtype Polymorphism

Subtype Polymorphism

- The same piece of code can be reused by taking different types of objects (as real method arguments), as long as they all belong to subtypes of a base type

Running Example



Upcasting and Downcasting

- Upcast: casting a subtype to a supertype
 - Always safe
- Downcast: cast a supertype to a subtype
 - Not always safe

```
ColoredCircle y = new ColoredCircle(2,red);  
Circle x = y;           // Upcast is always safe  
y = (ColoredCircle) x;  // Downcast is not safe  
                        // Java checks casts at run time
```

Field Allocation

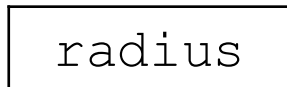
- Objects, with their fields and virtual table pointers, are allocated on the heap
- The layout for a field is the same as it is done for **structures** in **C** or other languages
 - The **fields** are allocated next to each other
 - Some **padding** may be required in between fields
 - Statically translate **field names** into relative addresses, with respect to the beginning of the object.
 - Fields for a **subclass** immediately follow the fields of the **superclass**

Memory Layout (Fields)

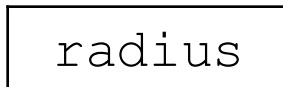
An object has

- fields (and ones from super class)

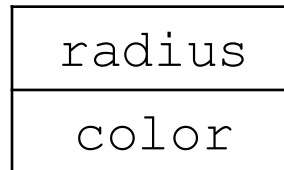
Circle **obj**



DoubleCircle **obj**



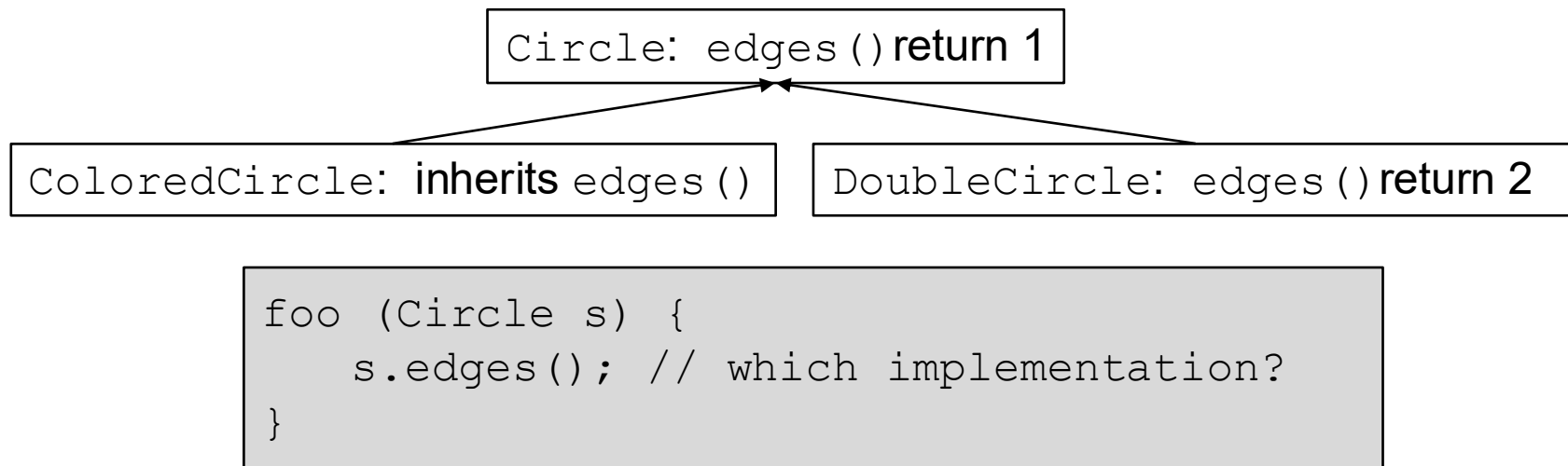
ColoredCircle **obj**



```
foo (Circle s) { s.radius; // offset? }
```

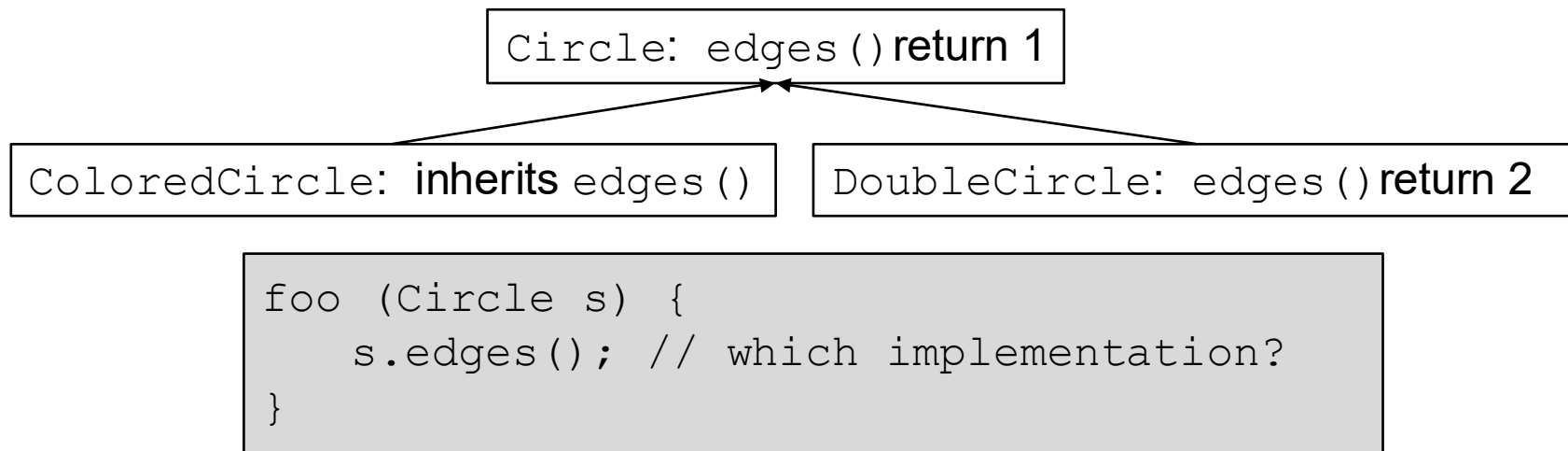
Caution: new fields in subtype should extend inherited fields due to subtype polymorphism

Memory Layout (Functions)



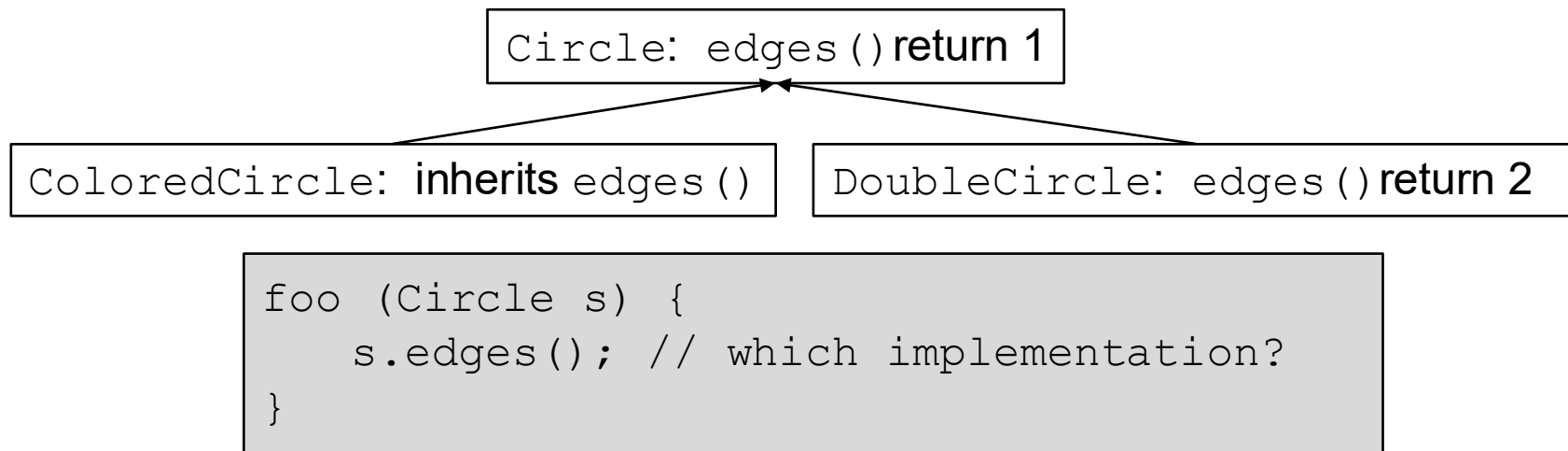
- **Static dispatch:** `s.edges ()` always returns 1
- **Dynamic dispatch:** return value of `s.edges ()` controlled by the type of `s`

Static Dispatch



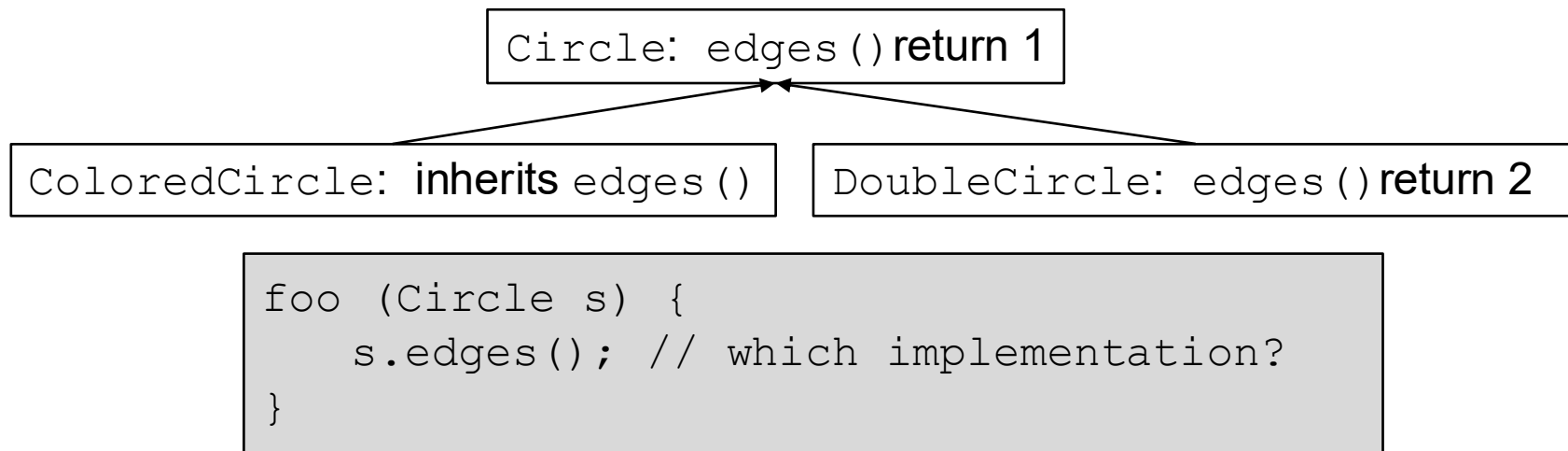
- Dispatch to the implementation in class `Circle`
 - Hence, `s.edges()` always returns 1

Static Dispatch



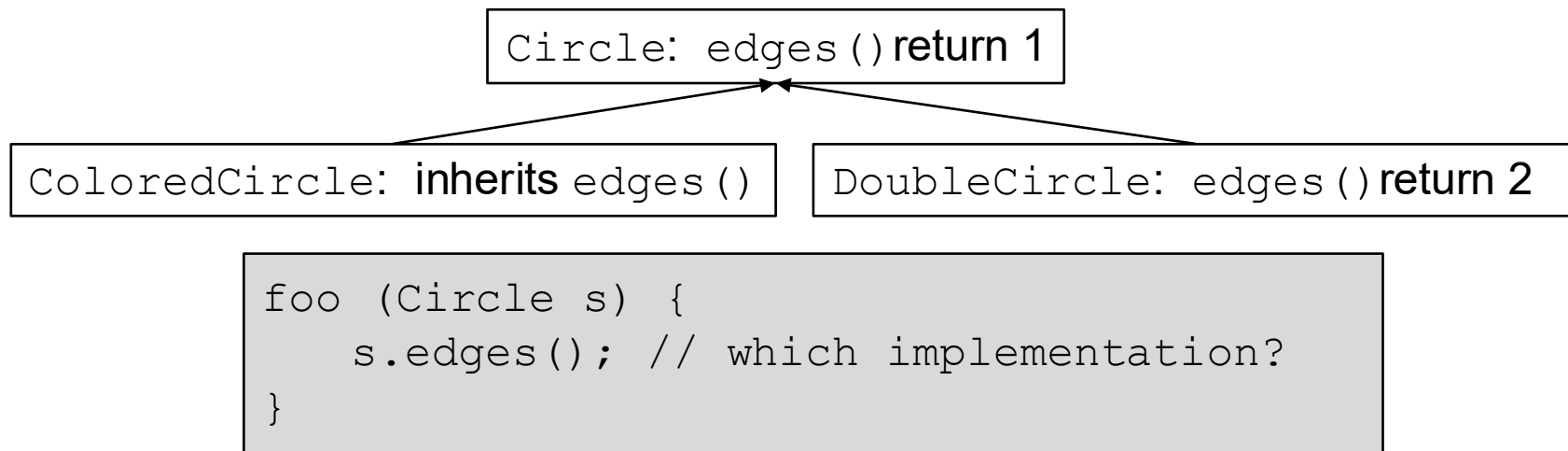
- The compiler can always tell which implementation at compile time, e.g., the `edges` method in class `Circle`

Dynamic Dispatch



- Dispatch to the implementation based on the type of the object *s*
 - Hence, *s.edges()* returns 2 when *s* is an object of class *DoubleCircle*

Dynamic Dispatch



The compiler does not know the type of `s`. How can it dispatch the method call to the correct implementation?

Trivial Memory Layout

An object has

- Fields (and ones from super class)
- Methods (and ones from super class)

Circle obj

radius
edges() :bin
area() :bin

DoubleCircle obj

radius
edges() :bin
area() :bin

ColoredCircle obj

radius
edges'() :bin
area() :bin

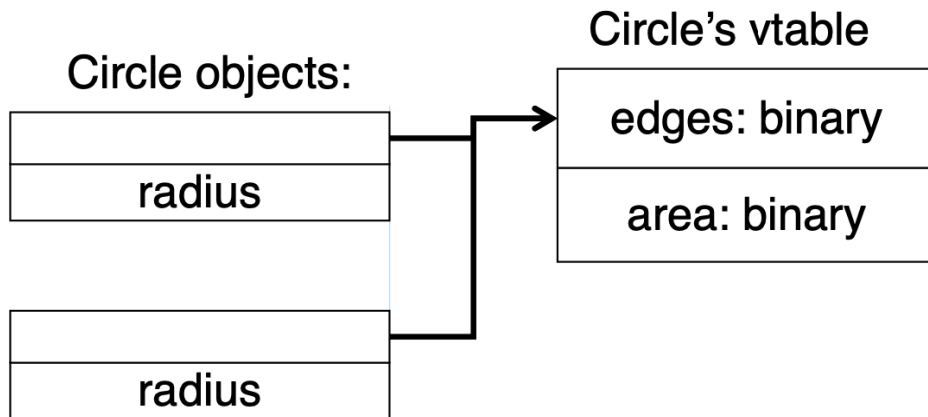
Issues:

- Each object has a copy of impl. code (waste of space)
- Polymorphic functions need to distinguish different layouts of classes (to find method offset)

Virtual Table (Tentative Design)

- A (shared) table containing method [binaries](#)

To save memory: one table per Class

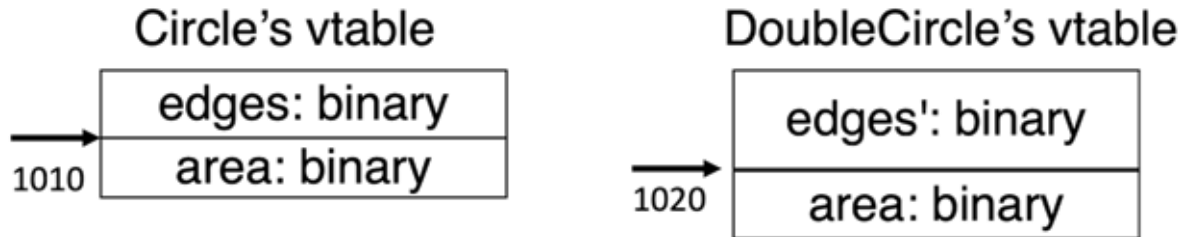


Fields stored
in each object

Points to methods stored
in one shared table

Virtual Table (Tentative Design)

- A (shared) table containing method [binaries](#)

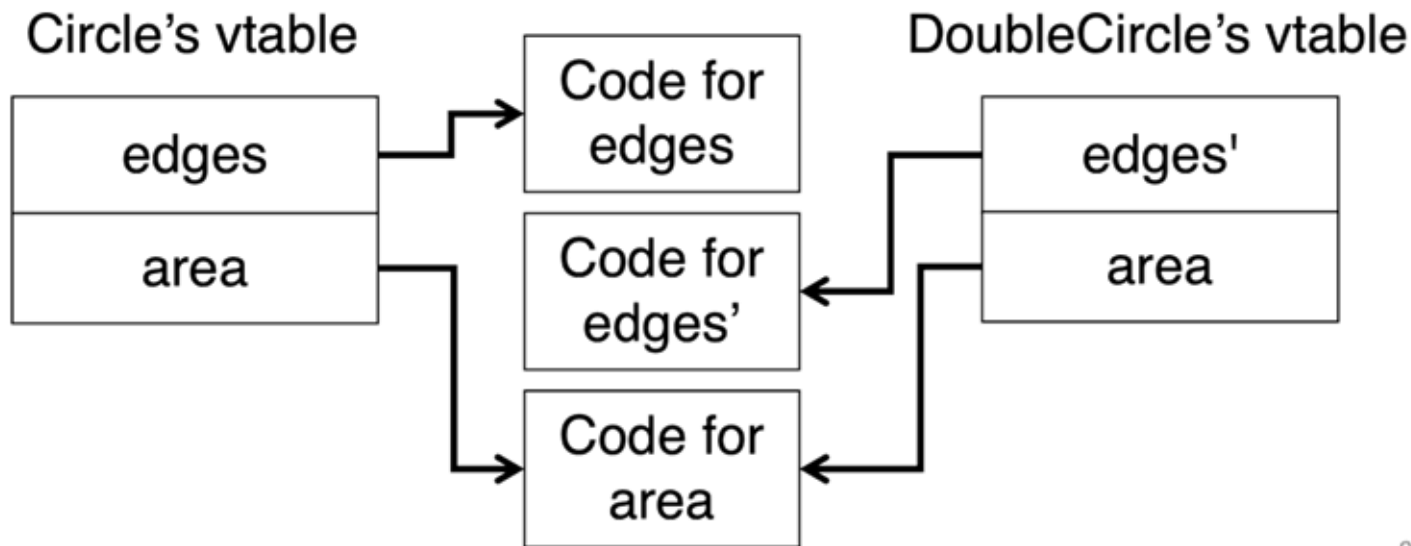


```
foo (Circle s) {  
    s.edges(); // code has different offsets  
}
```

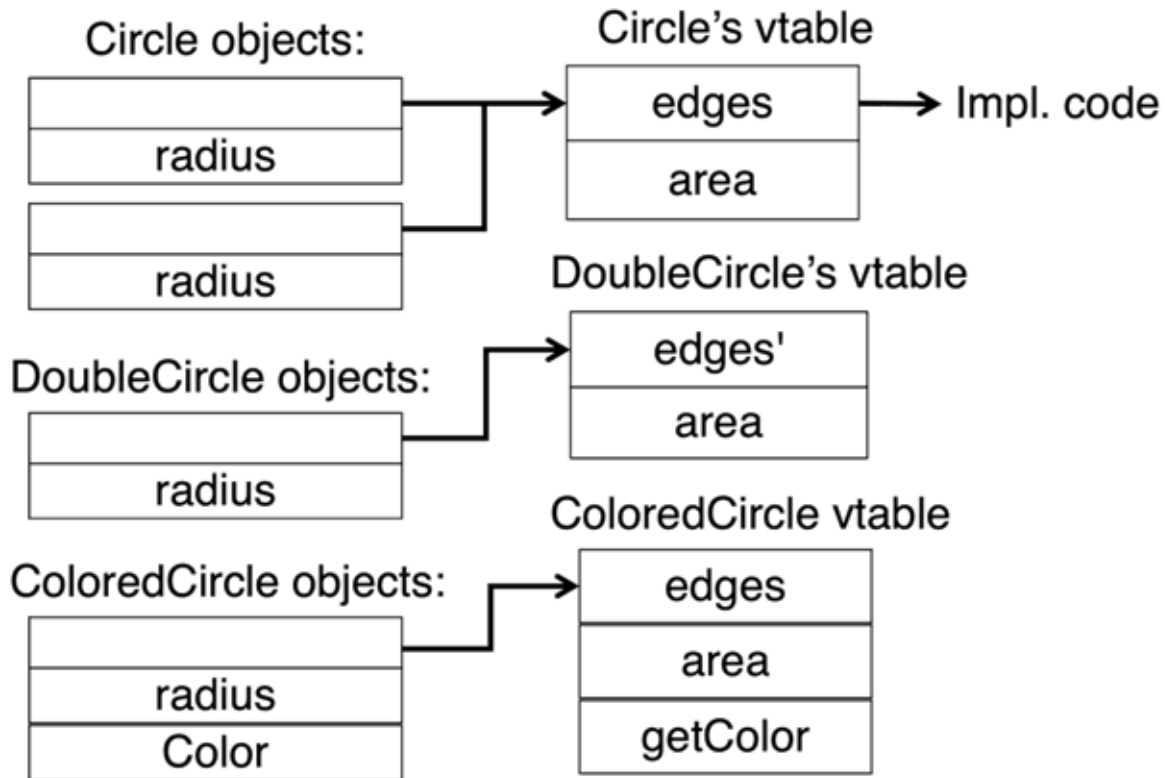
Issue: foo is compiled to different binaries with different offsets for different types of *s*

Virtual Table

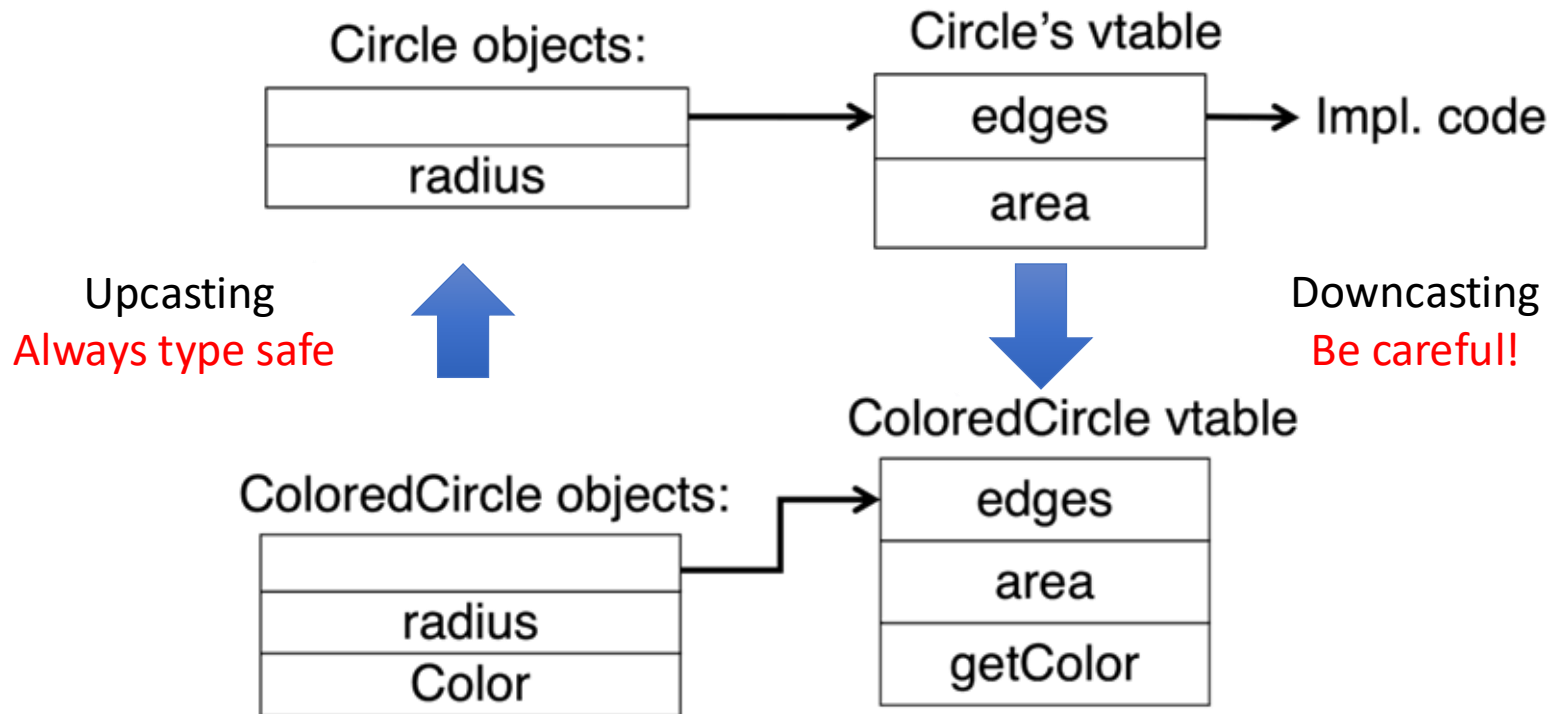
- A (shared) table containing [pointers](#) to methods



Virtual Table



Downcasting/Upcasting



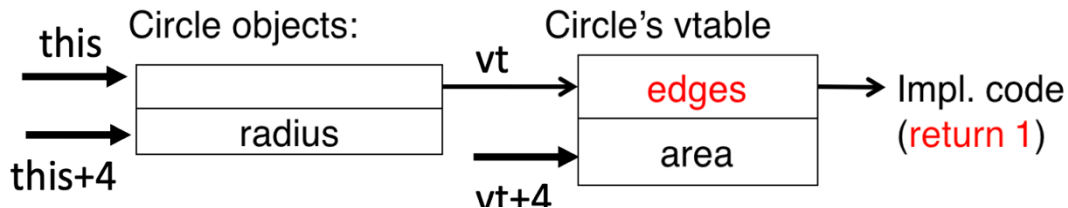
Member Lookup: Case 1

```
foo (Circle s) {  
    s.radius;  
    s.area();  
    s.edges();  
}
```



```
foo (Circle s) {  
    vt = *this;  
    *(this+4); //value of radius  
    call *(vt+4); // method area  
    call *vt; // method edges  
}
```

s is an object of class Circle



`s.edges()` returns 1

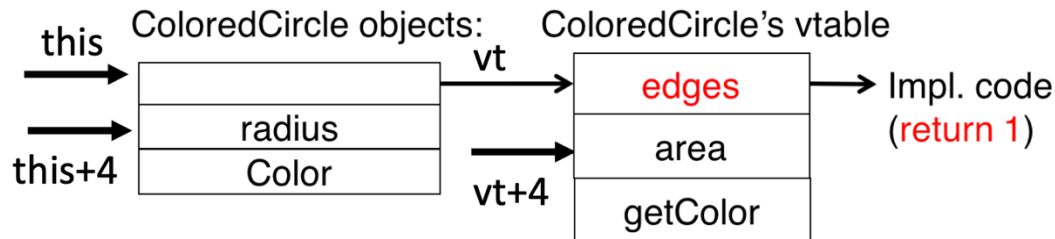
Member Lookup: Case 2

```
foo (Circle s) {  
    s.radius;  
    s.area();  
    s.edges();  
}
```



```
foo (Circle s) {  
    vt = *this;  
    *(this+4); //value of radius  
    call *(vt+4); // method area  
    call *vt; // method edges  
}
```

s is an object of class ColoredCircle



s.edges() returns 1
Upcasting is type safe

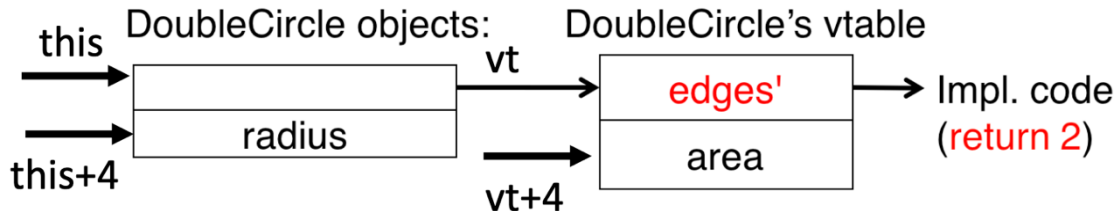
Member Lookup: Case 3

```
foo (Circle s) {  
    s.radius;  
    s.area();  
    s.edges();  
}
```



```
foo (Circle s) {  
    vt = *this;  
    *(this+4); //value of radius  
    call *(vt+4); // method area  
    call *vt; // method edges  
}
```

s is an object of class DoubleCircle



`s.edges()` returns 2

Dynamic Dispatch with VTables

```
foo (Circle s) {  
    s.radius;  
    s.area();  
    s.edges();  
}
```



```
foo (Circle s) {  
    vt = *this;  
    *(this+4); //value of radius  
    call *(vt+4); // method area  
    call *vt; // method edges  
}
```

One implementation
for all subtypes!

Static vs Dynamic Dispatching

Methods are dynamic

```
foo (Circle s) {  
    s.radius;  
    s.area();  
    s.edges();  
}
```

Methods are static

```
foo (Circle s) {  
    vt = *this;  
    *(this+4); //value of radius  
    call *(vt+4); // method area  
    call *vt; // method edges  
}
```

```
foo (Circle s) {  
    *(this+4); //value of radius  
    call Circle.area; // not in vt  
    call Circle.edges; // not in vt  
}
```


Static vs Dynamic Dispatching

- Static dispatch is easier to implement and more efficient
- Dynamic dispatch less efficient, but provides better extensibility (central to object-oriented programming)
 - Costs 10% to 20% extra overhead compared to a non-virtual function call

Dynamic Dispatch

- **Once Virtual, always virtual**
 - Once a base-class defines a function as virtual, it remains virtual through out the inheritance hierarchy
- C++: **static by default**, except the **virtual** methods
- Java: **dynamic by default**, expect the ones that cannot be overridden (e.g., final and static methods)
- Python: all methods use **dynamic** dispatch